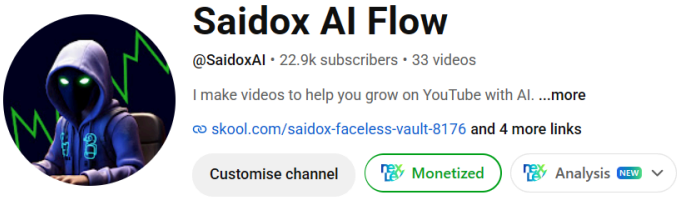
➔ **My Free Community**: *https://www.skool.com/saidox-faceless-vault-8176*

Tools and Resources:

- **Claude** free) prompting
- **Higgsfield AI:** Video Generator
- **Capcut:** Editing (free)

Created by:



Channel Cloner Master Prompt

Name: channel-research-and-video-generator

#. description:

Reverse-engineer any YouTube channel, then produce a brand-new video in that channel's style — end to end. The skill (1) asks for a channel, (2) deep-analyzes its TOP-performing videos (title patterns, thumbnail analysis with the channel's actual thumbnails pulled, hook structures, script format, pacing, story arc), (3) delivers the research as a downloadable, neutral HTML report, (4) offers 5–10 new titles to pick from, (5) writes a full script in the analyzed channel's style at a user-chosen length, (6) generates the voiceover and plays it back for approval, (7) presents a shot-by-shot table (timestamp, voiceover line, image idea, prompt) for approval, then generates images at a user-adjustable cadence (default one every 2.5s) plus a matching thumbnail in the channel's style, then (8) branches: animate the images into motion clips OR keep them as stills, and assembles everything synced to the VO, previewing for approval before the final render. Brand-agnostic and shareable — it detects which tools are on the machine and gracefully falls back to web search and prompt-only output when the research or generation tools are missing. Use when the user wants to analyze / reverse-engineer / "clone" a YouTube channel, study what makes a channel work, generate titles or scripts in a channel's style, or turn that research into a finished video with or without animation.

Channel Research and Video Generator

Give it a YouTube channel. It reverse-engineers what makes that channel work, then walks the user through producing a brand-new video in that channel's style: **analyze → report → title → script → voiceover → images → (optionally) animate → assembly → preview → render.**

This skill is brand-agnostic and meant to be shared. It hard-codes no one's brand. Its report and outputs use a neutral, dependency-light design by default; if the user wants their own branding, they can supply a logo and accent color and the skill will use those. It also **adapts to whatever tools are installed** — full generation when the research/gen tools are present, graceful web-search + prompt-only fallback when they're not.

What this is (and how it differs from a straight script-to-video pipeline)

- **Front half is the point:** it reverse-engineers a *real* channel (steps 1–3) before writing anything.
- **Script voice = the analyzed channel**, reproduced from its own hooks/structure/pacing — not any pre-existing house voice.
- **No storyboard sheets** — instead, a lightweight **shot-list table** you approve, then images at a **flat, user-adjustable cadence** (default one image every 2.5s).
- **Animate-or-not is a user choice, and it's the whole point.** Some videos just need images stitched to a voiceover; others want full motion. Both run from the same pipeline.

Step 0 — Capability check (run this FIRST, before Step 1)

Different machines have different tools connected. Detect what's available, tell the user which mode you're in, then route each phase to the best available path. **Never hard-fail because a paid tool is missing — fall back.**

Detection

Capability	Preferred	Detect by	Fallback chain
Channel research	Nexlev MCP (`mcp__4308…`) → vidiq MCP (`mcp__ff0f…`)	Are those MCP tools registered this session?	→ free `mcp__youtube-transcript__…` + `mcp__youtube__…` → **`WebSearch` + `WebFetch`** (universal) → the `watch` skill (yt-dlp + local whisper)
Voiceover	ElevenLabs (via a local TTS pipeline script if one exists in the repo)	Is a key / pipeline present?	→ HyperFrames local TTS (Kokoro, no key) → **prompt-only** (emit an ElevenLabs/TTS prompt block the user runs themselves)
Image generation	Higgsfield CLI (`higgsfield generate …`)	`higgsfield --version` via Bash, or MCP registered?	→ **prompt-only** (emit paste-ready image prompts for Midjourney / DALL·E / Nano Banana / any tool)
Animation (clips)	Higgsfield video (Seedance / Kling)	same as above	→ **prompt-only** (emit video prompts for Sora / Veo / Kling / Runway / Hailuo)

| **Assembly** | HyperFrames (`npx hyperframes …`) | `npx hyperframes --version` | → **assembly plan** the user runs in their own editor (Premiere / DaVinci / CapCut / iMovie) |
| **Duration probe** | `ffprobe` | `ffprobe -version` | → estimate from TTS metadata, flag as approximate |

Announce the mode in the first response, e.g. *"Research tools detected — I'll pull data directly."* or *"No YouTube research MCP here, so I'll research from the web."* and *"No Higgsfield on this machine — I'll hand you a prompt pack to run in your own image/video tools."*

Key point the user asked for: if the research tools aren't on the machine, **just research from the web** (`WebSearch` + `WebFetch` on the channel's pages). Thumbnails still work in any mode because YouTube serves them at a public URL (see Step 2). Whatever the mode, the *pipeline and the deliverables stay the same* — only the engine behind each step changes.

Pipeline overview

```
0.  Capability check → pick engines / fallbacks
1.  Ask for the channel
2.  Deep analysis of the channel's TOP performers
      (titles · thumbnails · hooks · script format · pacing · story arc)
3.  Deliver research as a neutral, downloadable HTML report
4.  Offer 5–10 new titles → user picks one
5.  Ask target script length (minutes)
6.  Write the script in the ANALYZED CHANNEL's style
7.  Generate the voiceover → PLAY IT for the user → APPROVE (locks total length) — before any images
8.  Ask animation style (3D Pixar / 2D / concept sketch / claymation / …) + optional custom style ref
9.  Shot-list table (timestamp · VO line · image idea · prompt) → APPROVE → generate images (one every 2.5s, ASK interval) + a THUMBNAIL in the channel's style
10. BRANCH → animate the images into clips?  YES → motion clips · NO → stills (ask static cuts vs Ken Burns)
11. Assemble the video, synced to the VO
12. Interactive HyperFrames PREVIEW (scrub / move things) → approve → final MP4 render → deliver every file location
```

Step 1 — Ask for the channel

The very first thing after the capability check:

> *"Which YouTube channel do you want me to analyze? Paste the channel URL, @handle, or name."*

Nothing else happens until the user provides one.

Step 2 — Deep analysis of the channel's TOP performers

Analyze the channel's **top-performing videos** (not just the latest). Extract: **title patterns**, **thumbnail patterns** (pull the actual thumbnails), **hook structures**, **script format**, **pacing**, **story arc**, and the core mechanism that makes it work.

Path A — research MCP present (Nexlev primary, vidiq fallback)

1. **Resolve** → `channel_resolver(input=<url/@handle>)` → `channelId`. (Or `youtube_channel_about(username=@handle)`.)
2. **Context** → `youtube_channel_about(channel_id)` — subs, niche, join date.
3. **Rank top performers** → `youtube_channel_outliers(channel_id, max_videos=200, min_outlier_threshold=2.0)`; take the top ~10 video IDs. Fallback: `youtube_channel_videos(channel_id, sort_by="popular")`.
4. **Titles + thumbnails + tags** → `youtube_video_details(video_id)` each, or one bulk `vidiq_get_videos_by_ids(videoIds[≤50])` (5 cr).
5. **Transcripts** → `get_bulk_video_transcripts(videoIds[≤10])`. Free per-video fallback: `mcp__youtube-transcript__get_transcript(url)`.
6. **Audience** → `youtube_video_comments(video_id, sort_by="top")` on the top 3–5.
7. Optional thumbnail quantification → `get_similar_thumbnails` / `vidiq_score_thumbnail` (5 cr).

> vidiq tools cost 5 credits each — batch (`vidiq_get_videos_by_ids`), don't loop. Nexlev `get_bulk_video_transcripts` caps at 10 IDs.

Path B — no research MCP → research from the web

1. **Find the channel + its popular videos** with `WebSearch` (e.g. *"<channel> most popular videos"*) and `WebFetch` on `https://www.youtube.com/@<handle>/videos`. This is less precise than the outlier ranking — say so, and lean on view counts / "most popular" sorting where visible.
2. **Video IDs** come from the watch URLs (`…/watch?v=<ID>`).
3. **Transcripts** → `mcp__youtube-transcript__get_transcript(url)` if present; otherwise the **`watch` skill** (yt-dlp + captions, local whisper fallback), or `WebFetch` a transcript mirror. If none work, analyze from titles + thumbnails + descriptions and flag that transcripts were unavailable.

Thumbnails work in EVERY mode (pull and actually look at them)

Thumbnails are served publicly — no API needed. For each video ID:

1. Download `https://i.ytimg.com/vi/<videoId>/maxresdefault.jpg` (fall back to `hqdefault.jpg`) to the scratchpad with `curl`.
2. **`Read`** the local images so vision can inspect them.
3. Analyze the cluster: face vs no-face, color temperature, text load, focal contrast, emotion, number/arrow motifs.

Reserve `watch_youtube_video_and_ask` (Nexlev, expensive) or the `watch` skill's frame extraction only for genuinely visual/pacing questions the above can't answer.

Synthesize

Title patterns · thumbnail patterns · hook structures (first ~15s) · script format (segments, retention devices, CTA placement) · pacing & story arc · **the one replicable mechanism**.

Step 3 — Deliver the research as a downloadable HTML report (neutral / brand-agnostic)

Write a self-contained `research-report.html` into the project folder and open it. **The default design is deliberately neutral** — a clean, print-friendly editorial report using neutral fonts (Fraunces + Inter, with a system fallback) and a single configurable accent color. **No specific brand is baked in.**

Optional user branding: if the user wants their own look, ask for (a) an accent color hex and (b) an optional logo image, and set `--accent` / drop the logo in the header. Otherwise use the neutral default below. Do **not** read or apply any repo brand file — this skill is shared and must not assume one house style.

Sections: **Title Patterns · Thumbnail Analysis (gallery grid of the pulled thumbnails) · Hook Structures · Script Format · Pacing & Story Arc · What Makes It Work.** Include a **"Download as document"** button that calls `window.print()` with a print stylesheet (saves a clean PDF).

Neutral report shell (paste-ready — fill the `id`-tagged elements; drop thumbnail URLs into `.thumb img`)

```html
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Channel Research Report</title>
<link rel="preconnect" href="https://fonts.googleapis.com"><link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,500;9..144,600;9..144,700&family=Inter:wght@400;500;600&display=swap" rel="stylesheet">
<style>
:root{
  --bg:#f6f7f9; --bg-2:#eef0f5; --surface:#ffffff;
```

```
  --ink:#14161c; --muted:#5f6875; --line:#e7e9ef;
  --accent:#5b57e0; --accent-2:#8b5cf6; --accent-soft:rgba(91,87,224,.09); --on-accent:#fff;
/* accent is user-overridable */
  --radius:18px; --pad:34px; --shadow:0 1px 3px rgba(20,22,28,.05),0 18px 40px -24px
rgba(20,22,28,.28);
  --font-ui:'Inter',system-ui,-apple-system,"Segoe UI",Roboto,Helvetica,Arial,sans-serif;
  --font-head:'Fraunces',Georgia,"Times New Roman",serif;
}
*{box-sizing:border-box;margin:0}
body{font-family:var(--font-ui);color:var(--ink);font-size:16.5px;line-height:1.62;-webkit-font-s
moothing:antialiased;
  background:radial-gradient(1000px 520px at 82% -8%,var(--accent-soft),transparent
60%),linear-gradient(180deg,var(--bg),var(--bg-2));
  min-height:100vh;padding:44px 22px}
.wrap{max-width:980px;margin:0 auto}
h1,h2,h3{font-family:var(--font-head);letter-spacing:-.01em;line-height:1.15;font-weight:600}
.mono{font-family:ui-monospace,"SF
Mono",Menlo,Consolas,monospace;text-transform:uppercase;letter-spacing:.16em;font-size:
11.5px;font-weight:600;color:var(--accent)}
.card{background:var(--surface);border:1px solid
var(--line);border-radius:var(--radius);padding:var(--pad);margin-bottom:20px;box-shadow:va
r(--shadow)}
.hero{position:relative;overflow:hidden}
.hero::before{content:"";position:absolute;inset:0 0 auto
0;height:5px;background:linear-gradient(90deg,var(--accent),var(--accent-2))}
.hero-top{display:flex;align-items:flex-start;gap:20px;margin:6px 0 22px}
.hero-top .logo{height:52px;width:auto;border-radius:10px}
.hero-top .meta{flex:1}.hero h1{font-size:38px;margin:8px 0}
.hero .sub{color:var(--muted);font-size:16px}
.btn-print{font-family:var(--font-ui);font-size:14px;font-weight:600;cursor:pointer;color:var(--on
-accent);

background:linear-gradient(135deg,var(--accent),var(--accent-2));border:none;border-radius:
11px;padding:12px 20px;
  box-shadow:0 8px 20px -8px var(--accent);white-space:nowrap}
.btn-print:hover{filter:brightness(1.06)}
.stats{display:flex;flex-wrap:wrap;gap:12px}
.stat{flex:1;min-width:130px;background:var(--bg);border:1px solid
var(--line);border-radius:12px;padding:12px 14px}
.stat
.k{font-family:ui-monospace,monospace;font-size:11px;letter-spacing:.1em;text-transform:up
percase;color:var(--muted)}
.stat .v{font-family:var(--font-head);font-size:22px;font-weight:600;margin-top:2px}
.note{font-size:12.5px;color:var(--muted);background:var(--surface);border:1px solid
var(--line);border-left:3px solid var(--accent);
  border-radius:10px;padding:13px
16px;margin-bottom:24px;line-height:1.55;box-shadow:var(--shadow)}
.sec-h{display:flex;align-items:center;gap:14px;margin-bottom:16px}
```

```
.badge{flex:none;width:38px;height:38px;border-radius:11px;background:var(--accent-soft);color:var(--accent);

font-family:ui-monospace,monospace;font-weight:700;font-size:15px;display:flex;align-items:center;justify-content:center}
.sec-h h2{font-size:22px}
p{margin-bottom:12px}p:last-child{margin-bottom:0}.lede{font-size:17.5px}
ul{margin:6px 0 12px 20px}li{margin-bottom:7px}strong{color:var(--ink);font-weight:600}
.tag{display:inline-block;font-family:ui-monospace,monospace;font-size:11.5px;color:var(--accent);
  background:var(--accent-soft);border-radius:999px;padding:6px 13px;margin:4px 6px 4px 0}
.pull{border-left:3px solid var(--accent);padding:10px 0 10px 18px;margin:14px 0;font-family:var(--font-head);font-size:17px;font-style:italic}
.pull .src{display:block;font-family:var(--font-ui);font-size:13px;font-style:normal;color:var(--muted);margin-top:6px}
.formula{background:linear-gradient(180deg,var(--accent-soft),transparent);border:1px solid var(--line);border-radius:12px;padding:16px 18px;margin-top:12px}
.formula code{font-size:13px}
.gallery{display:grid;grid-template-columns:repeat(auto-fill,minmax(240px,1fr));gap:15px;margin-top:16px}
.thumb{position:relative;border-radius:13px;overflow:hidden;border:1px solid var(--line);background:var(--bg);box-shadow:var(--shadow);transition:transform .18s ease,box-shadow .18s ease}
.thumb:hover{transform:translateY(-3px);box-shadow:0 1px 3px rgba(20,22,28,.06),0 22px 44px -20px rgba(20,22,28,.4)}
.thumb .im{position:relative}
.thumb img{width:100%;aspect-ratio:16/9;object-fit:cover;display:block}
.thumb .views{position:absolute;right:8px;bottom:8px;background:rgba(10,12,18,.82);color:#fff;font-family:ui-monospace,monospace;font-size:11px;padding:3px 8px;border-radius:6px}
.thumb .cap{font-size:12.5px;color:var(--muted);padding:10px 12px;line-height:1.4}.thumb .cap b{color:var(--ink)}
@media (max-width:560px){.hero h1{font-size:30px}}
@media print{
  @page{margin:13mm}
  body{background:#fff;padding:0;font-size:12px}.btn-print{display:none!important}
  .card{box-shadow:none!important;border:1px solid #dcdfe6!important;break-inside:avoid}
  .hero::before,.badge{-webkit-print-color-adjust:exact;print-color-adjust:exact}
  .thumb{box-shadow:none!important}.gallery{grid-template-columns:repeat(3,1fr)}
}
</style></head>
<body><div class="wrap">
  <header class="card hero">
    <div class="hero-top">
      <!-- optional: <img class="logo" src="user-logo.png" alt=""> -->
      <div class="meta">
```

```
      <div class="mono">Channel Deep-Dive</div>
      <h1 id="channelName">@ChannelName</h1>
      <div class="sub" id="channelMeta">Niche · analyzed YYYY-MM-DD</div>
    </div>
    <button class="btn-print" onclick="window.print()">Download as document</button>
  </div>
  <div class="stats" id="stats">
    <div class="stat"><div class="k">Subscribers</div><div class="v">—</div></div>
    <div class="stat"><div class="k">Total views</div><div class="v">—</div></div>
    <div class="stat"><div class="k">Videos analyzed</div><div class="v">—</div></div>
    <div class="stat"><div class="k">Top video</div><div class="v">—</div></div>
  </div>
 </header>
 <div class="note" id="method">METHOD — …</div>
 <section class="card"><div class="sec-h"><div class="badge">01</div><h2>Title
Patterns</h2></div>
  <p id="titlePatterns"></p><div><span class="tag">tag</span></div>
  <div class="formula"><span class="mono">Title templates</span> …</div></section>
 <section class="card"><div class="sec-h"><div class="badge">02</div><h2>Thumbnail
Analysis</h2></div>
  <p id="thumbNotes"></p><div class="gallery" id="thumbGallery">
    <figure class="thumb"><div class="im"><img src="" alt=""><span
class="views"></span></div><figcaption class="cap"></figcaption></figure>
  </div></section>
 <section class="card"><div class="sec-h"><div class="badge">03</div><h2>Hook
Structures</h2></div><p id="hooks"></p></section>
 <section class="card"><div class="sec-h"><div class="badge">04</div><h2>Script
Format</h2></div><p id="scriptFormat"></p></section>
 <section class="card"><div class="sec-h"><div class="badge">05</div><h2>Pacing &amp;
Story Arc</h2></div><p id="pacing"></p></section>
 <section class="card"><div class="sec-h"><div class="badge">06</div><h2>What Makes It
Work</h2></div><p id="whatWorks"></p></section>
</div></body></html>
```

Also give a short chat summary, but the HTML report + its "Download as document" button is the primary deliverable.

Step 4 — Offer new titles

Ask if they want new titles built off the identified patterns. Generate **5–10 options** that reproduce the channel's title formula (structure, length, hook type, emotional angle) applied to the user's topic. Number them; let the user **pick one** (or request another round). Mirror the target channel's title conventions; if the user has platform-specific policies of their own, they can flag them.

Step 5 — Ask target script length

> *"How many minutes should the script be?"*

Length drives VO duration and ultimately the image count. Lock a number before writing.

Step 6 — Write the script in the ANALYZED channel's style

Write the full script from the **extracted channel formula** — the reverse-engineered hooks, structure, pacing, and cold-open pattern from Step 2 — applied to the chosen title at the chosen length.

> **Do not route this through any pre-existing house/brand voice engine.** The purpose is to reproduce the *analyzed channel's* style; a fixed house-voice writer would overwrite it. The analyzed channel's patterns ARE the template. Match its opening-hook template, segment cadence, retention devices, CTA placement, and words-per-minute pace.

Show the script for approval before generating the voiceover.

Step 7 — Voiceover generation

Generate the full VO first — it **locks total duration**, which the image count in Step 9 reads from (never estimate from word count).

- **Voice:** ask the user for their preferred voice (e.g. an ElevenLabs voice ID) or use a default storyteller voice. If the repo has a reusable TTS pipeline script, reuse its `generate_voiceover(text, out_path)` function rather than calling the API fresh; otherwise call the TTS engine directly.
- **No key / no ElevenLabs?** Fall back to HyperFrames local TTS (Kokoro), or, in prompt-only mode, emit a TTS prompt block (script text + voice/speed/emotional direction) for the user to run themselves.
- **Speed:** narration/explainer **1.0–1.05**; cinematic/storyteller **0.85–0.90** — match the analyzed channel's pace.
- **Probe real duration** with `ffprobe` (source of truth). Save to `clips/voiceover.mp3`.

▸ VOICEOVER APPROVAL GATE (mandatory). Play the finished VO back for the user and get **explicit approval before generating any images**. Every image is timed and matched to this exact audio, so changing the voice or pacing *after* the images exist means re-timing (and often re-rolling) the whole set. Iterate here first: wrong timbre → try another voice; too fast/slow → adjust speed and re-render; a flubbed word or bad emphasis → tweak the text/punctuation and re-render. **Do not proceed to Step 8/9 (style + images) until the user approves the voiceover.**

Step 8 — Animation style + optional custom style reference

Ask what look they want, offering a list:

- **3D Pixar** · **2D animation** · **concept sketch** · **claymation** · (plus room for others — anime, watercolor, live-action cinematic, etc.)

The user may also **hand over their own style reference image** to lock to. If they do, that image becomes the style anchor (Step 9) and every generation matches it.

Also lock the format + quality here

Ask two quick questions now — they set every image AND the final composition size:

- **Aspect ratio** — `16:9` (default, standard YouTube) · `9:16` (Shorts / Reels / TikTok) · `1:1` (square) · `4:5` (feed). This also sets the composition dimensions in Step 11 (16:9 → 1920×1080, 9:16 → 1080×1920).
- **Resolution / quality** — `2k` (default: faster, cheaper, great for most videos) or `4k` (crisper, more detail, slower and more credits — worth it for premium or large-screen work). On GPT Image 2 this rides with `--quality high`; Nano Banana Pro has no separate quality flag (resolution only).

Carry the chosen `<ASPECT>` and `<RES>` into every command in Step 9 and the composition size in Step 11.

Style blocks (inject verbatim into every image prompt for the chosen style):

- **3D Pixar:** `Pixar-style 3D animated render, soft global illumination, subsurface-scattered skin, rounded stylized proportions, large expressive eyes, warm cinematic key light, shallow depth of field, polished CGI, family-film look.`
- **2D animation:** `hand-drawn 2D animation, clean bold ink outlines, flat cel-shaded color fills, simple shapes, limited palette, TV-anime/cartoon look, crisp vector-like edges, no photoreal texture.`
- **Concept sketch:** `loose concept-art pencil sketch, graphite and charcoal linework, gestural cross-hatching, visible construction lines, monochrome with light tonal shading, unfinished sketchbook feel, matte paper texture.`
- **Claymation:** `stop-motion claymation, sculpted modeling-clay characters, visible fingerprints and tool marks, matte plasticine surface, slightly uneven handmade forms, soft studio lighting, tactile miniature-set look (Aardman-style).`

Step 9 — Generate images (no storyboards)

Interval — ask before generating. Default is **one image every 2.5s** of VO, but 2.5s is a lot of images for a long video. Ask:

> *"I'll generate one image every 2.5 seconds by default — that's ~N images for your Ns voiceover. Want to widen the interval (e.g. every 4s or 5s) to use fewer images?"*

Image count = `ceil(VO_seconds / interval)`.

Consistency method (no storyboard sheet to carry style)

1. **Generate ONE style-anchor image first** in the chosen style; get a quick thumbs-up.
2. **Pass that anchor as a reference to every subsequent generation** so all images inherit its look, AND prepend the **same style block** (Step 8) to every prompt.
3. Keep each prompt tight (`<~200 tokens`) — the prompt describes *what to render*, not what's in the reference.

Build the shot list — show it, get approval BEFORE generating

A lightweight **text** plan (not a storyboard image). It's the map the user tweaks *before* any credits are spent, and the reference they use to request changes *afterward* ("re-roll shot 14"). Always do this **between** the style-anchor approval and mass generation.

Slice the voiceover/script into **one row per image** at the chosen interval (default 2.5s), and present this table **inline in the conversation** for approval:

Shot	Timestamp	Voiceover line	Image idea	Scene prompt
1	0:00–0:02.5	the words narrated over this beat	one-line description of the visual	the scene-specific prompt appended to the shared style block
2	0:02.5–0:05	…	…	…

Rules:
- State the **shared STYLE BLOCK once** above the table — the constant style + recurring-character wrapper (from Step 8 / the approved anchor). Each row's **Scene prompt** is only the *variable* part, so the full generation prompt = STYLE BLOCK + Scene prompt. This keeps rows short and every shot trivially tweakable.
- Wardrobe / setting / props / **era** live in the **Scene prompt**, not the style block — a historical topic says "animal-hide wrap, savanna"; a modern topic says "hoodie, phone, kitchen." Same art style, different world.
- **Present it inline — do NOT write it to a separate file** unless the user asks.
- Let the user **edit any row** (reword a prompt or image idea), **add / remove / merge shots**, or change the interval, before anything is generated.
- Only after the user **approves the table** do you generate (next). **Number the output files to the shot numbers** (`img1.png` = Shot 1) so later re-rolls map cleanly to rows.

If Higgsfield is present (CLI via Bash)

```bash
```

DEFAULT — GPT Image 2 (<ASPECT> and <RES> come from Step 8, e.g. 16:9 and 2k or 4k)
higgsfield generate create gpt_image_2 --prompt "<STYLE BLOCK> + <scene from script beat>" \
  --image ./images/style-anchor.png --aspect_ratio <ASPECT> --resolution <RES> --quality high --wait

BACKUP — Nano Banana Pro (model id = nano_banana_2 ; NOT nano_banana_flash)
higgsfield generate create nano_banana_2 --prompt "<STYLE BLOCK> + <scene>, match the reference style/palette/linework exactly" \
  --image ./images/style-anchor.png --aspect_ratio <ASPECT> --resolution <RES> --wait
```

- **GPT Image 2** default: use the chosen `<RES>` (`2k` or `4k`) and `<ASPECT>` from Step 8 with `--quality high`. Full ranges — resolution `{1k,2k,4k}`, quality `{low,medium,high}`, aspect `{1:1,4:3,3:4,16:9,9:16,3:2,2:3}`.
- **Nano Banana Pro (`nano_banana_2`)** backup — same `<RES>`/`<ASPECT>`, no `--quality` flag; it locks a reference style harder, so switch to it if the look drifts.
- Save to `images/img1.png … imgN.png` mapped to script beats in order.

### If Higgsfield is NOT present → prompt-only

Emit N paste-ready image prompts (one per beat), each with: recommended tool (Midjourney / DALL·E / Nano Banana / Stable Diffusion), the chosen aspect ratio + target resolution, the style block, the scene, and the style-anchor instruction ("match the attached reference exactly"). The user generates in their own tool and drops the files into `images/`.

### Hands & character consistency (front-load when people appear, any mode)

- **ABSOLUTE HAND ANATOMY:** `FIVE digits = four fingers + one thumb, count 1,2,3,4,5; not three, not four total, not six; no fused or melted fingers. HAND CHECK wherever hands show.`
- **SINGLE-CHARACTER CONSISTENCY:** declare the reference the EXACT and ONLY source of truth, list the locked features, forbid beautify / slim / enlarge-eyes / stylistic drift / skin-tone change; close-ups render the SAME face at closer range.

### Also generate a THUMBNAIL — in the analyzed channel's style

The thumbnail is a huge part of why a channel gets clicks, and Step 2 already reverse-engineered its formula. Generate a matching one for the new video as part of this stage.

1. **Pull the formula from Step 2 / the report** — the channel's locked thumbnail recipe: focal subject, character emotion, composition (e.g. two-character face-off), palette, and especially the **text treatment**.
2. **Write the on-thumbnail hook text in the channel's convention** — most channels do NOT put the full title on the thumbnail. Match what they actually do (e.g. Mack = a **2–3
```

word ALL-CAPS yellow-with-black-outline** fragment/question like "WHY WHITE?"). Draft 2–3 hook-text options.
3. **Render 2–3 thumbnail variations** so the user can pick. Always **16:9** (YouTube thumbnails are 16:9 regardless of the video's aspect ratio) at `--resolution 2k` (or `4k`) `--quality high`. Render in the video's chosen visual style / custom style reference so the thumbnail matches the film, but follow the **channel's** thumbnail composition + text formula.

```bash
higgsfield generate create gpt_image_2 --prompt "<CHANNEL THUMBNAIL FORMULA applied to this video> — <focal character + big emotion>, <composition>, <palette>. Bold 2-3 word hook text '<HOOK>' in <channel's text style>. <STYLE BLOCK or match reference>." \
  --image ./images/style-anchor.png --aspect_ratio 16:9 --resolution 2k --quality high --wait
```

4. If the baked-in text renders messy, generate the scene **clean** and composite the hook text as a crisp overlay (HyperFrames or an image editor) instead of relying on the model to spell it.
5. If vidiq is available, optionally `vidiq_score_thumbnail(title, image)` for a CTR read on each variation.
6. Save as `images/thumbnail-1.png … thumbnail-3.png`; the picked one becomes `images/thumbnail.png` and ships in the final deliverables.

> Prompt-only mode: emit the thumbnail prompt + the 2–3 hook-text options for the user's own image tool.

Step 10 — BRANCH: animate the images, or keep them as stills?

Before any video generation, ask:

> *"Do you want to animate these images into motion clips, or keep them as still images sequenced to the voiceover?"*

YES → animate

Turn the stills into motion via **image-to-video** (Higgsfield Seedance 2.0 / Kling if present; otherwise **prompt-only** video prompts for Sora / Veo / Kling / Runway / Hailuo). Each still is the clip's start frame; sequence the clips in assembly (**Mode C** below).

- **Duration floor:** Seedance clips run 4–15s. If the image interval is under 4s, either **group consecutive stills into 4–6s motion segments** or **offer to widen the interval** so each image maps cleanly to one clip. Flag this rather than silently up-sampling.

NO → stills only, then **ask how they should move**

> *"Static cuts, or add a bit of motion (slow Ken Burns zoom/pan with crossfades)?"*

- **Static cuts** → Mode A. · **A bit of motion** → Mode B (Ken Burns).

Either way, the voiceover carries the timing.

Step 11 — Assemble the video

If HyperFrames is present, use the `hyperframes` + `hyperframes-cli` skills and build `index.html`. Write a **minimal, neutral `DESIGN.md`** noting the visual identity lives in the generated frames/clips (no brand file references). If HyperFrames is NOT present, output an **assembly plan** (clip/image order, per-item durations, crossfade specs, audio-mix targets) the user runs in their own editor.

Root shell (all modes):
```html
<div id="root" data-composition-id="root" data-start="0" data-duration="TOTAL"
    data-width="1920" data-height="1080">
```
Register `window.__timelines["root"] = tl;` with `gsap.timeline({paused:true})`.
`data-track-index` groups overlap detection only; CSS `z-index` controls visual layering.

Mode A — stills, static cuts
`count = ceil(VO / interval)`; image *i* → `data-start = i*interval`, `data-duration = interval` (last image extends to cover any VO remainder). Each image gets its own `data-track-index`.
```html
<img id="im0" class="still" data-start="0"   data-duration="2.5" data-track-index="0"
src="images/img1.png">
<img id="im1" class="still" data-start="2.5" data-duration="2.5" data-track-index="1"
src="images/img2.png">
```
```css
.still{position:absolute;inset:0;width:100%;height:100%;object-fit:cover;opacity:0}
```
```js
const IV=2.5, N=Math.ceil(VO/IV);
for(let i=0;i<N;i++){const s=i*IV;
  tl.to(`#im${i}`,{opacity:1,duration:0.12,ease:"power1.out"},s);
  if(i<N-1) tl.to(`#im${i}`,{opacity:0,duration:0.12,ease:"power1.in"},s+IV);}
```

Mode B — stills, Ken Burns + crossfade
Overlap neighbors by `X=0.6s`; each `data-duration = IV + X`. **Wrap the img and animate the wrapper — never the media element itself.**
```html
```

<img src="images/img1.png">
```
```css
.kb{position:absolute;inset:0;overflow:hidden;opacity:0;will-change:opacity}
.kb img{width:100%;height:100%;object-fit:cover;transform-origin:center;will-change:transform}
```
```js
const IV=2.5, X=0.6;
imgs.forEach((_,i)=>{const s=i*IV, dur=IV+X;
  tl.fromTo(`#kb${i} img`,{scale:1.0,x:0,y:0},{scale:1.12,x:i%2?-30:30,y:-20,duration:dur,ease:"none"},s);
  tl.to(`#kb${i}`,{opacity:1,duration:X,ease:"power1.inOut"},s);
  if(i<imgs.length-1) tl.to(`#kb${i}`,{opacity:0,duration:X,ease:"power1.inOut"},s+IV);});
```

Mode C — video clips + VO
Each video on its own `data-track-index`, `muted playsinline`, `object-fit:cover`, `opacity:0`. 0.5s crossfade; each clip starts 0.5s before the prior ends. **VO outruns clips:** extend the final clip's `data-duration` past its source so the video freezes on its last frame while VO finishes (never time-stretch).

Audio tracks
```html
<audio id="vo"    data-start="0" data-duration="<VO>" data-track-index="20" data-volume="1.0"  src="clips/voiceover.mp3"></audio>
<audio id="a1"    data-start="0" data-duration="15"   data-track-index="10" data-volume="0.35" src="clips/clip1.mp4"></audio> <!-- foley, animate path -->
<audio id="music" data-start="X" data-duration="Y"    data-track-index="21" data-volume="0.13" src="clips/music.mp3"></audio> <!-- optional, sparse -->
```
VO `1.0`; per-clip foley `0.30–0.40`; sparse music `0.13` (0.22 fights the VO), only under a climax if used.

Lint gotchas (`npx hyperframes lint` before every render)
- Each video/audio source needs its **own** `data-track-index` — overlapping same-track = error. Convention: images/videos 0–5(+), foley 10–15, VO 20, music 21.
- Root needs `data-composition-id`; the timeline must be registered on `window.__timelines[...]`. Video must be `muted playsinline`; sound is always a separate `<audio>`.
- Don't animate a media element's dimensions (animate a wrapper). No `Math.random()`, no `repeat:-1`. Sparse-keyframe warnings on generated MP4s are safe to ignore.

Step 12 — Preview in HyperFrames → approve → final render → deliver
```

The preview is the interactive HyperFrames studio — NOT a rendered MP4. The whole point is that the user can scrub the timeline, watch it play, and move things around *before* committing to a render. An MP4 is something you *watch*; the HyperFrames preview is something you can *change*.

1. `npx hyperframes lint` — fix all errors first.
2. **Open the interactive preview:** `npx hyperframes preview --port <port>` and give the user the local URL. This launches the HyperFrames studio — they scrub the timeline, play it, and see exactly what the final will look like. If they want changes (re-time a shot, swap an image, adjust a crossfade, move a beat, retime to the VO), **edit `index.html`** and the preview hot-reloads. Iterate live here. **Do NOT render an MP4 for the preview step.**
3. **Get explicit approval in the preview.** Prompt for the specific things to check: shot order, timing against the VO, Ken Burns feel, crossfades, any drift. Keep iterating on `index.html` until they approve.
4. **Only after approval → render the MP4:** `npx hyperframes render --quality high --fps 24 --output renders/final.mp4` (24fps cinematic; 30fps snappy/social).
5. **Deliver a full LOCATION MAP so the user can find and edit everything themselves.** List each deliverable as its **full ABSOLUTE path on the user's machine** — the direct location they can paste into File Explorer / Finder or double-click — **not** a relative path. Resolve `<PROJECT>` to the real absolute project folder first (e.g. `C:\Users\<user>\...\<project>` on Windows, `/Users/<user>/.../<project>` on Mac):
   - **Final video** → `<PROJECT>\renders\final.mp4`
   - **Thumbnail** → `<PROJECT>\images\thumbnail.png`
   - **Editable composition** → `<PROJECT>\index.html` (re-open anytime with `npx hyperframes preview`, then re-render)
   - **All images** → `<PROJECT>\images\` (img1…imgN, the style anchor, the references)
   - **Voiceover** → `<PROJECT>\clips\voiceover.*`
   - **Script** → `<PROJECT>\SCRIPT.md`
   - **Research report** → `<PROJECT>\research-report.html`
   - **Whole project folder** → `<PROJECT>\`

   Where the host renders clickable file links, keep the **absolute path as the visible text** so the user can still read exactly where it lives on disk. State plainly: the video is assembled from these pieces, so they can swap any image, re-record the VO, or edit `index.html` and re-render — nothing is baked in. (In prompt-only mode, deliver the prompt pack + assembly plan instead of a rendered MP4.)

Project folder structure

```
<project-name>/
  research-report.html      # neutral, downloadable channel analysis (Step 3)
  DESIGN.md                 # thin, neutral — notes identity lives in the frames/clips
  index.html                # assembly composition (HyperFrames mode)
  PROMPTS-PACKAGE.md        # the deliverable in prompt-only mode
  images/
    style-anchor.png        # the locked style reference
```

```
    img1.png … imgN.png     # generated stills (2.5s cadence)
    thumbnail.png           # picked thumbnail in the channel's style
  clips/
    voiceover.mp3           # VO
    clip1.mp4 …             # motion clips (animate path only)
    music.mp3               # optional sparse bed
  renders/
    v1-draft.mp4 → final.mp4
```

Handling variations

- **No research tools on the machine:** research from the web (Step 2, Path B) — thumbnails still work via the public `i.ytimg.com` URLs.
- **No generation tools:** run prompt-only — emit image + video + TTS prompts and an assembly plan the user runs in their own tools.
- **Vertical (Shorts/Reels):** image `--aspect_ratio 9:16`, composition `1080×1920`, vertical Ken Burns framing.
- **Very short (<15s):** a handful of images; single-pass assembly. **Long (10min+):** confirm a wider interval up front (2.5s would be hundreds of images).
- **Style drift in images:** switch to `nano_banana_2` (harder style lock), or train a reusable character reference.
- **User wants their own branding on the report:** collect an accent hex + optional logo and set `--accent` / the header logo — otherwise keep the neutral default. Never assume a house brand.